

TCP Window Scaling Attack

CSE 406 Security Project

Rageeb Hasan Shafee
2105175

Intesar Tahmid Alam
2105180

Hillol Chakma
2105181

September 19, 2026

1 Introduction

TCP Window Scale is used to allow TCP connections to work with windows larger than the normal 16-bit TCP window. The scale value is negotiated during the initial TCP handshake using the SYN and SYN-ACK packets.

In this project, we studied what happens when an on-path attacker changes the Window Scale value in the client's SYN packet. The main idea is simple: if the client and server use different Window Scale values, they can have different ideas about the actual receive window.

We tested three attacks:

1. **Deflate:** change the client's Window Scale from 10 to 0. The server then thinks the client's window is much smaller.
2. **Inflate:** change the client's Window Scale to 14. The server then thinks the client's window is much larger than the client's real receive buffer.
3. **Slow Read / Zero-Window DoS:** do not modify packets. Instead, keep many connections open and read data very slowly.

We also tested three defenses:

1. Static ARP entries
2. IPsec AH
3. A live Window Scale mismatch detector

For each experiment, we compare what we expected with what actually happened.

2 How the Whole Experiment Works

Before looking at the individual attacks, it is useful to understand the overall experiment.

The normal traffic path is:

Client → Attacker → Server

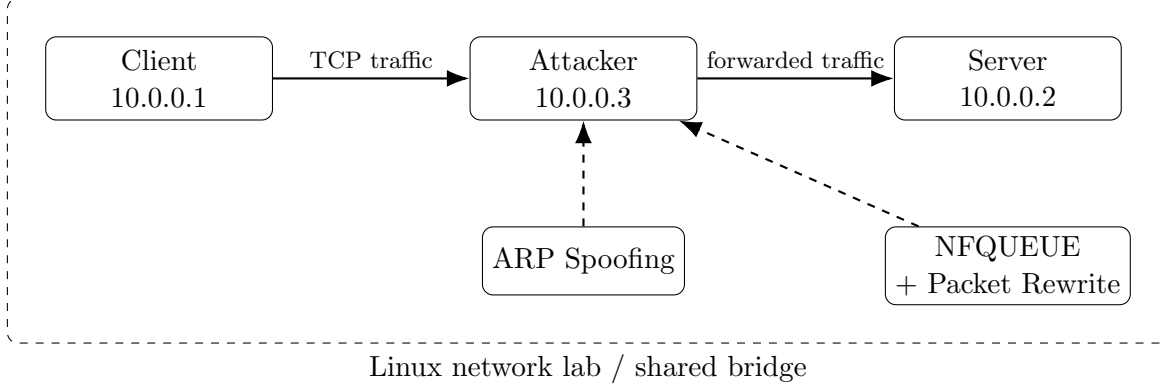


Figure 1: Basic experimental setup

The attacker first uses ARP spoofing to become a man-in-the-middle. After that, selected TCP packets can be inspected and modified.

3 TCP Window Scale in Simple Terms

The TCP receive window contains a 16-bit advertised value. Window Scale allows this value to represent a much larger window.

The effective window can be written as:

$$EffectiveWindow = AdvertisedWindow \times 2^{WindowScale}$$

For example, with Window Scale 10:

$$2^{10} = 1024$$

So the advertised window is multiplied by 1024.

The important point for this project is that the Window Scale value is negotiated during the initial handshake.

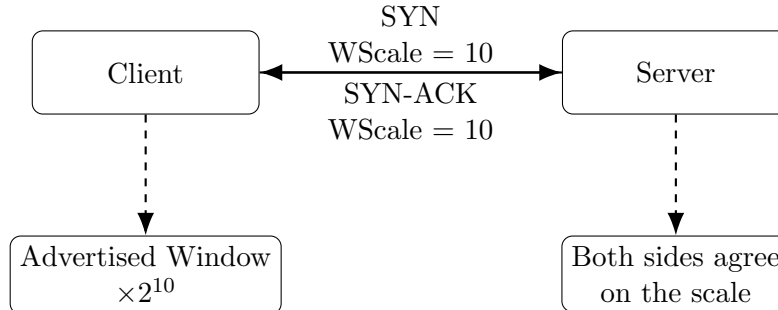


Figure 2: Normal TCP Window Scale negotiation

An attacker who changes the Window Scale value in the SYN can make the server use a different scale from the one actually used by the client.

4 Lab Environment

The experiments were performed in a small Linux network lab using network namespaces. Three namespaces were connected through a Linux bridge.

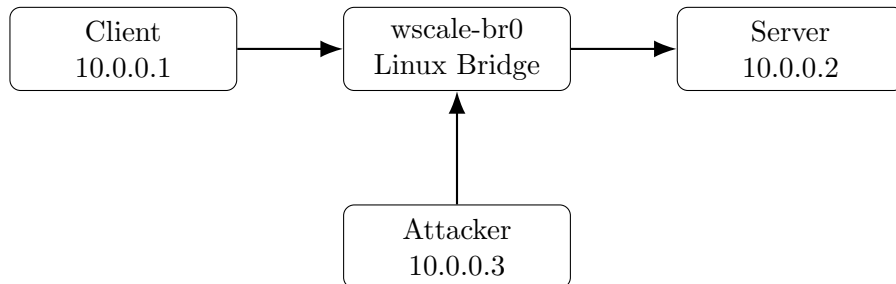


Figure 3: Three-node Linux namespace lab

The roles were:

- **Client:** 10.0.0.1
- **Server:** 10.0.0.2
- **Attacker:** 10.0.0.3

The attacker used ARP spoofing to become a man-in-the-middle (MITM). It then used `iptables`, `NFQUEUE`, `Scapy`, and `NetfilterQueue` to inspect and modify selected TCP SYN packets.

The main tools used were:

- `arp_spoof.py` for ARP poisoning
- `wscale_attack.py` for changing Window Scale
- `detect_wscale_mismatch.py` for offline detection
- `live_ids.py` for live detection
- `tcpdump` for packet capture
- Wireshark for manual packet inspection

TCP performance was measured using Linux’s `TCP_INFO` socket option. This provided throughput, retransmissions, RTT, congestion window, and Window Scale information.

5 Attack 1: Deflate

5.1 Idea

In the deflate attack, the attacker changes the client’s Window Scale from 10 to 0.

The Window Scale calculation is:

$$EffectiveWindow = AdvertisedWindow \times 2^{WindowScale}$$

Therefore, changing the scale from 10 to 0 makes the server believe that the client’s window is

$$2^{10} = 1024$$

times smaller than it really is.

5.2 Deflate Attack Diagram

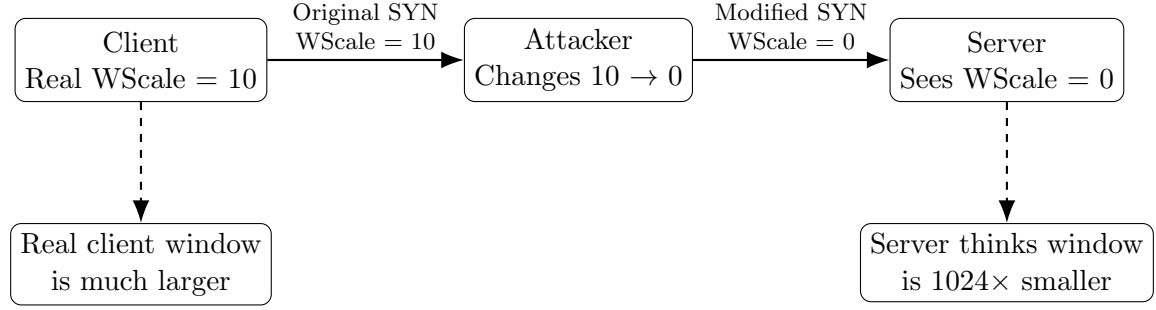


Figure 4: Deflate attack: Window Scale is reduced from 10 to 0

5.3 Attack Process

The experiment was performed in the following steps.

1. First, a normal 20 MB transfer was performed without an attacker. This gave us the baseline result.
2. ARP spoofing was started so that the attacker could stay between the client and server.
3. IP forwarding was enabled on the attacker.
4. TCP SYN packets were sent to NFQUEUE using:

```
iptables -t mangle -A FORWARD -p tcp --tcp-flags SYN SYN \
-j NFQUEUE --queue-num 0 --queue-bypass
```

5. `wscale_attack.py` inspected the client's SYN packet and changed its Window Scale from 10 to 0.
6. The same 20 MB transfer was performed again.
7. Finally, the packet capture was checked using the detector script and Wireshark.

5.4 Expected Result

We expected the server to believe that the client's receive window was much smaller than its actual window.

Because the scale changed from 10 to 0, the server should limit its sending rate heavily.

We therefore expected a large throughput reduction without a large number of retransmissions.

5.5 Actual Result

The attack successfully changed the Window Scale:

```
2026-09-19 00:20:48,239 listening on NFQUEUE 0
2026-09-19 00:20:50,989 rewrote WScale
10.0.0.1:44992 -> 10.0.0.2:9000
shift 10 -> 0
```

The measured results were:

Run	Throughput (MB/s)	Retransmissions	RTT (μ s)	WScale
Baseline	1781.163	0	360	10
Deflate	2.314	0	9	0

Table 1: Deflate attack results

The throughput dropped by about 99.9%.

There were no retransmissions. This shows that the slowdown was caused by TCP flow control and the incorrect Window Scale value, rather than packet loss.

The packet detector also found the expected mismatch:

```
10.0.0.1:44992 -> 10.0.0.2:9000 [SYN]
wscale values seen: [0, 10]

10.0.0.2:9000 -> 10.0.0.1:44992 [SYN-ACK]
wscale values seen: [10]

996 anomalously small effective windows
```

5.6 What We Learned

The main prediction was correct.

One interesting observation was the very large number of packets. An early uncapped capture contained more than 1.3 million packets for a single 20 MB transfer.

This makes sense because the effective window became approximately 63 bytes. The transfer therefore required a very large number of small data segments and acknowledgements. Since the attacker was between the two endpoints, each packet also crossed the bridge twice.

Another observation was that the server's congestion window stayed around 10 segments. The connection was receive-window-limited, so congestion control did not have much opportunity to increase the congestion window.

6 Attack 2: Inflate

6.1 Idea

The inflate attack is the opposite of deflate.

Instead of reducing the Window Scale, we increase it to 14.

For example, if the real scale is 10 but the server sees 14, the server thinks the client's window is

$$2^{14-10} = 16$$

times larger than it really is.

If the client's real receive buffer is small, the server may send more data than the client can actually hold. This can lead to retransmissions and stalls.

6.2 Inflate Attack Diagram

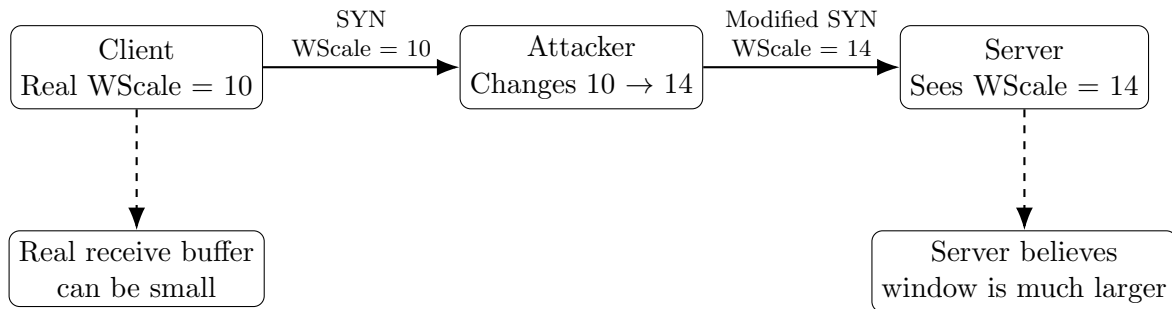


Figure 5: Inflate attack: the server is given an inflated Window Scale

6.3 Attack Process

The same MITM and NFQUEUE pipeline used in the deflate attack was used.

The main change was:

```
--new-shift 14
```

The first experiment used the normal Linux receive-buffer behaviour.

After that experiment showed almost no effect, the client's receive buffer was fixed at 65536 bytes:

```
throughput/bulk_client.py --rcvbuf 65536
```

This prevented Linux from automatically increasing the receive buffer.

6.4 Expected Result

We expected the server to believe that the client's receive window was much larger than its real buffer.

This should cause the sender to transmit too much data, resulting in retransmissions and possibly stalls.

6.5 First Attempt

The Window Scale rewrite was successful:

```
shift 10 -> 14
```

However, there was almost no negative effect.

Throughput changed from:

1176.326 MB/s

to:

2480.288 MB/s

and retransmissions remained at zero.

So, the first attempt did not match our prediction.

6.6 Why the First Attempt Failed

The main reason was the lab environment.

The network was extremely fast and had almost no delay. Linux also used receive-buffer autotuning and increased the client's real buffer quickly.

The entire 20 MB transfer finished in less than a second. Therefore, the server's false window was not large enough, for long enough, to create a visible problem.

To make the attack effect easier to observe, we fixed the client's receive buffer at 65536 bytes.

6.7 Second Attempt

After fixing the receive buffer, the attack produced the expected effect.

Run	Throughput	Retrans.	cwnd	Rewrite	Real Shift
1st	47.354 MB/s	306	156	10→14	10
2nd	22.747 MB/s	1065	121	10→14	10
3rd	13.535 MB/s	1398	167	1→14	1

Table 2: Inflate attack with a fixed receive buffer

The third run also showed:

```
10.0.0.1:59258 -> 10.0.0.2:9000 [SYN]
wscale values seen: [1, 14]

advertised_window=0
wscale=14
effective=0 bytes
```

6.8 Another Observation

The real Window Scale changed from 10 to 1 between repeated runs.

This happened because Linux can keep TCP metrics for previously used destinations. Earlier degraded connections affected the parameters chosen for later connections.

As a result, the difference between the real and fake scale became:

$$2^{14-10} = 16$$

in one run and:

$$2^{14-1} = 8192$$

in another run.

This explains why the retransmissions increased from 306 to 1065 and then to 1398.

6.9 What We Learned

The inflate attack is more dependent on the endpoint's TCP behaviour than the deflate attack.

In our lab, the attack was not clearly visible until the client's receive buffer was explicitly limited.

This is an important experimental finding: the attack itself can be implemented correctly, but the final effect depends on factors such as receive-buffer autotuning, network speed, and TCP metrics.

7 Attack 3: Slow Read / Zero-Window DoS

7.1 Idea

The Slow Read attack works differently.

It does not modify packets or use ARP spoofing.

Instead, the attacker opens many connections and reads the response very slowly. Each connection uses a very small receive buffer.

In this experiment, the server had a worker pool of only 20 connections.

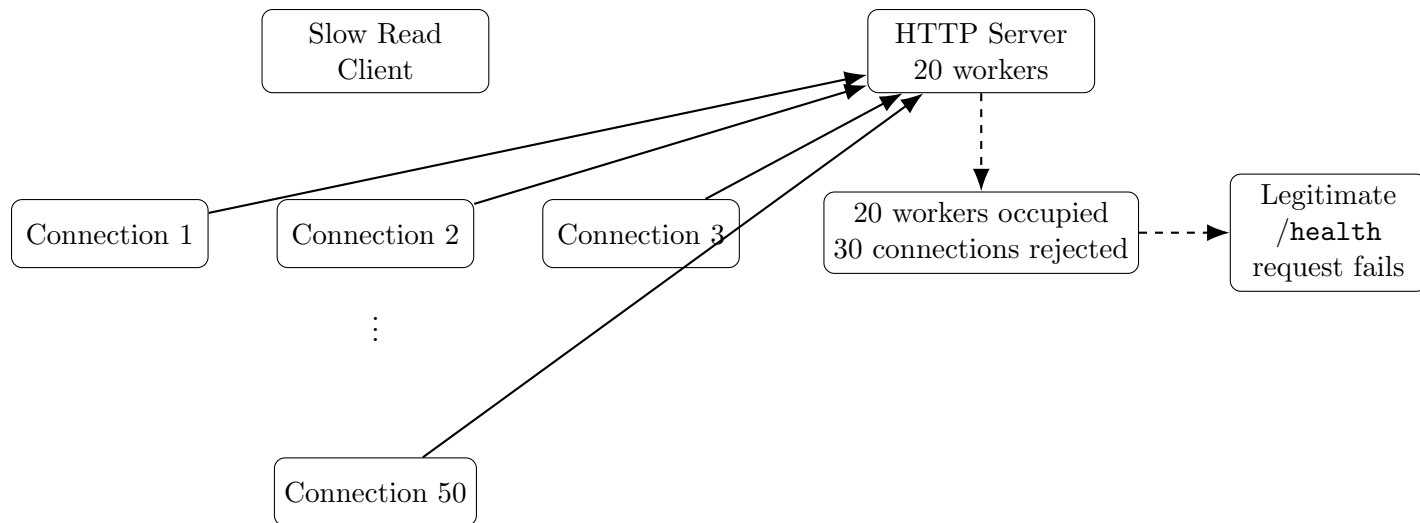


Figure 6: Slow Read attack and connection-pool exhaustion

7.2 Slow Read Attack Diagram

7.3 Attack Process

The server was started with:

```
--max-conns 20
```

The attacker then:

- opened 50 connections,
- requested `/bigfile`,
- used an 8-byte receive buffer,
- read only a few bytes every second,
- kept the connections open for 30 seconds.

The server also provided a `/health` endpoint so that we could test whether a normal client could still access the server.

7.4 Expected Result

There were 50 attack connections but only 20 available worker slots.

Therefore, we expected the worker pool to become full and legitimate requests to fail or stall during the attack.

After the attack connections closed, the server should recover.

7.5 Actual Result

The results were:

```
Baseline health check
-> ok=True rtt=0.001s

Health check during attack
-> ok=False rtt=3.000s
    Connection reset by peer

20/50 connections held open

Health check after attack
-> ok=True rtt=0.001s
```

Exactly 20 connections stayed open for the full 30 seconds.

The other 30 connections were rejected or reset because the worker pool was already full.

The health check failed during the attack and immediately recovered after the connections were released.

7.6 Result

The result matched our prediction.

This attack was more predictable than the inflate attack because it did not depend on Window Scale manipulation or receive-buffer autotuning. It only required the server to have a limited resource per connection.

8 Defense Mechanisms

We tested three defenses against the attack pipeline:

1. Static ARP entries
2. IPsec AH
3. Live Window Scale mismatch detection



Figure 7: Three defense approaches used in the project

The goal was not only to describe the defenses, but to run them against the same lab setup.

8.1 Defense A: Static ARP Entries

How It Works

The attack starts with ARP poisoning.

Static ARP entries prevent the attacker from changing the client and server's ARP mappings.

The script:

```
defense/harden_arp.sh
```

reads the real MAC addresses and creates permanent neighbour-table entries.

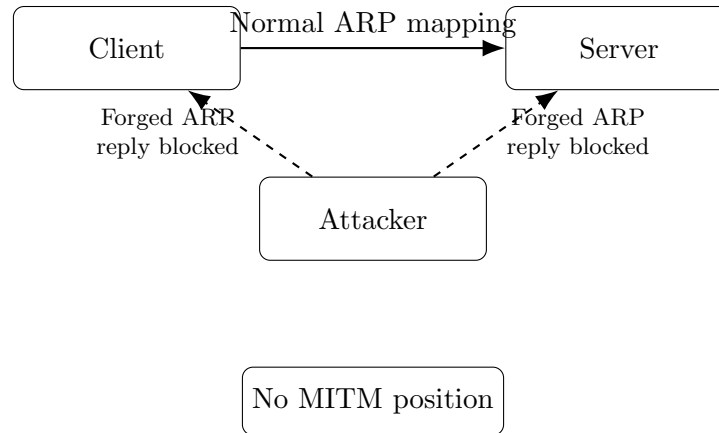


Figure 8: Static ARP prevents the attacker from becoming MITM

Expected Result

The forged ARP replies should be ignored.

Therefore:

- the attacker should not become MITM,
- no packets should reach the attacker's rewrite script,
- no Window Scale should be changed,
- throughput should remain close to baseline.

Actual Result

The rewrite program started, but it did not rewrite any packet:

```
2026-09-19 02:01:56,938 listening on NFQUEUE 0  
(no further lines -- zero rewrites)
```

The packet capture showed:

```
10.0.0.1:53470 -> 10.0.0.2:9000 [SYN]
wscale values seen: [10]

10.0.0.2:9000 -> 10.0.0.1:53470 [SYN-ACK]
wscale values seen: [10]

0 flow(s) with a WScale mismatch
```

The transfer also completed normally:

```
server: throughput=2196.388 MB/s
total_retrans=0
snd_wscale=10

client: throughput=1827.000 MB/s
total_retrans=0
snd_wscale=10
rcv_wscale=10
```

Conclusion

The defense worked as expected.

The attacker could not obtain the MITM position, so the later Window Scale modification step could not happen.

8.2 Defense B: IPsec AH

How It Works

IPsec Authentication Header (AH) was configured in transport mode.

The setup used Linux's `ip xfrm` interface and a manually configured shared HMAC-SHA1-96 key.

AH provides integrity protection for the IP packet, including the TCP header.

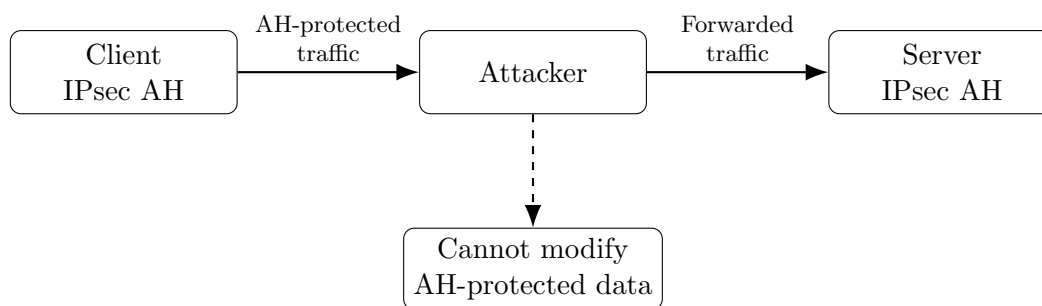


Figure 9: IPsec AH protects packet integrity

The configuration was applied using:

```
defense/setup_ipsec_ah.sh
```

Static ARP protection was not enabled for this experiment, so the attacker could still remain on-path.

Expected Result

We expected the attacker to be unable to modify the protected packet.

Two possible outcomes were considered:

1. The modified packet could fail AH verification and be dropped.
2. The attacker's TCP-specific NFQUEUE rule could stop matching because AH changes the IP protocol from TCP (6) to AH (51).

In either case, the Window Scale should not be modified.

Actual Result

The IPsec configuration was installed successfully.

However, the attacker logged zero rewrites:

```
2026-09-19 02:23:40,801 listening on NFQUEUE 0
(no further lines -- zero rewrites)
```

The detector found no SYN packets in the capture:

```
=== SYN / SYN-ACK WScale values per flow ===
(empty)

0 flow(s) with a WScale mismatch
```

The transfer completed successfully:

```
server: throughput=158.140 MB/s
total_retrans=202
snd_wscales=10

client: throughput=156.081 MB/s
total_retrans=0
snd_wscales=10
```

Why the Result Was Different from the Prediction

The connection did not time out.

Instead, the second expected outcome happened: the attacker's TCP-based NFQUEUE rule did not match the AH-wrapped packets.

Therefore, the attacker's rewrite program never received the packets.

The Window Scale remained unchanged.

The lower throughput and retransmissions observed during this run were associated with the heavy workload on the single physical machine. The capture reported 5,587 dropped packets from its own kernel buffer while around 9,000 packets were received.

Importantly, this experiment did not directly test AH's cryptographic rejection of a modified packet. The attacker never reached the point where it could modify the AH-protected packet.

Conclusion

IPsec AH prevented the Window Scale tampering in this experiment. The exact mechanism was that the attacker's TCP packet filter did not match the AH-wrapped traffic.

8.3 Defense C: Live Window Scale Monitor

The third defense is detection rather than prevention.

The detector compares Window Scale values observed in duplicate SYN packets. It also checks for unusually small effective TCP windows.

The live detector is:

```
defense/live_ids.py
```

A reference Zeek implementation was also included:

```
defense/wscale_ids.zeek
```

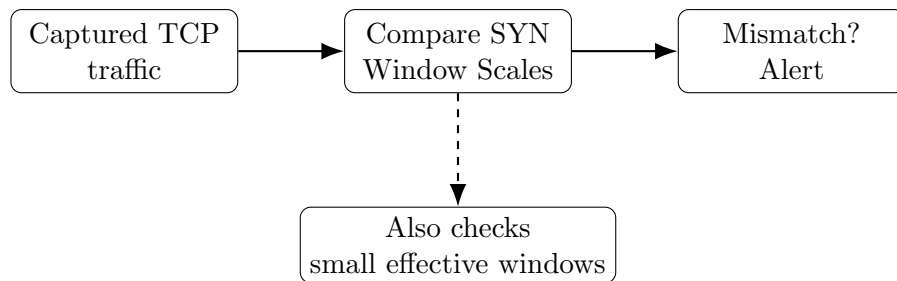


Figure 10: Live IDS detection process

Zeek itself was not executed in this lab because it was not available through the system's default package repositories.

Result

The live detector successfully detected the modified SYN.

It produced:

- 1 Window Scale mismatch alert
- 24,891 anomalous-window alerts
- 0 false positives in the tested data

The single mismatch corresponded to the one SYN packet that was actually modified by the attack.

9 Expected vs. Actual Workflow

A useful part of this project was comparing the expected behaviour with what actually happened.

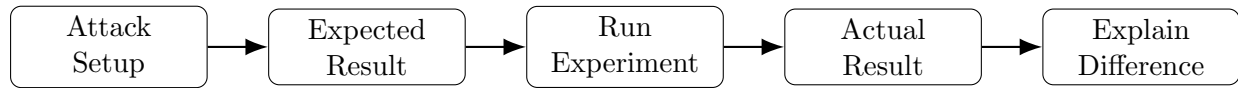


Figure 11: Experimental workflow used in the report

This process was especially important for the inflate attack and IPsec AH experiment, where the observed result was different from the original prediction.

10 Assumptions

The experiments depend on several assumptions.

1. **The attacker is on-path.** The attacker must be on the same Layer-2 network as the client and server. ARP spoofing is assumed to be possible.
2. **There is no transport-layer integrity protection.** The Window Scale attack assumes that TCP headers are not protected by IPsec AH, ESP, or another integrity mechanism.
3. **The attacker uses one network interface.** The attacker relies on Linux IP forwarding and ARP resolution to relay packets.
4. **Window Scale is negotiated during the handshake.** The attack depends on the scale value being selected during the SYN/SYN-ACK exchange and not being re-checked later.
5. **Both endpoints support Window Scale.** If Window Scale is not used, the normal 64 KB TCP window is used and there is no scale value to modify.
6. **Linux TCP behaviour can change between runs.** Receive-buffer autotuning and TCP metrics caching affected the inflate experiment.
7. **The lab is much faster than a normal network.** The experiments used namespaces and a software bridge on one physical machine. Therefore, the exact throughput and timing should not be treated as real-WAN measurements.
8. **Only the client's SYN was modified.** The experiments used the `client-syn` target. The software also supports other targets, but they were not used here.
9. **Slow Read requires a limited server resource.** The Slow Read attack worked because the server had a fixed 20-connection worker pool.
10. **Static ARP requires trusted MAC addresses.** The correct MAC addresses must be known before the attacker successfully poisons the ARP cache.
11. **The IPsec experiment used a manual shared key.** This demonstrates the integrity mechanism, not a complete production key-management system. A real deployment would normally use IKE for key negotiation and rotation.
12. **The IDS is detection-only.** The live detector reports suspicious traffic but does not automatically block or rate-limit it.

11 Overall Results

11.1 Attack Results

Attack	Expected	Actual	Result
Deflate	Large throughput drop with no packet loss	99.9% throughput drop, 0 retransmissions, Window Scale mismatch	Matched
Inflate, normal buffer	Retransmissions and stalls	No visible damage; throughput increased and retransmissions were 0	Did not match
Inflate, fixed buffer	Retransmissions and stalls	306–1398 retransmissions across repeated runs	Matched
Slow Read	Connection pool exhaustion	20/50 connections occupied the pool and health checks failed	Matched

Table 3: Summary of attack experiments

The deflate and Slow Read attacks produced their expected results during the first properly executed experiments.

The inflate attack needed one change to the experimental setup: the client’s receive buffer had to be fixed at 65536 bytes.

This difference was mainly caused by Linux receive-buffer autotuning and the very fast lab network.

11.2 Defense Results

Defense	Main Mechanism	Observed Result
Static ARP	Prevents ARP poisoning	Attacker could not become MITM; 0 rewrites
IPsec AH	Provides integrity protection	0 rewrites and 0 Window Scale mismatches
Live IDS	Detects Window Scale inconsistencies	1 mismatch alert and 24,891 anomalous-window alerts

Table 4: Summary of defense experiments

All three defenses were tested against the attack pipeline.

Static ARP stopped the attack before the attacker could become man-in-the-middle.

IPsec AH prevented the attacker’s packet modification pipeline from operating on the protected traffic.

The live IDS detected the Window Scale mismatch.

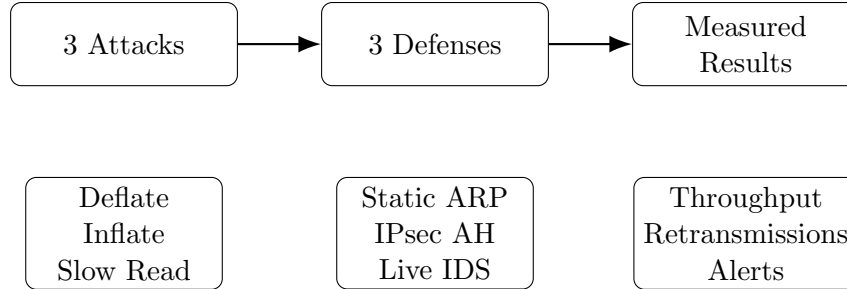


Figure 12: Overall project structure

12 Conclusion

This project demonstrated three different TCP-based denial-of-service scenarios in a controlled Linux lab.

The deflate attack showed that changing the Window Scale from 10 to 0 can make the server severely underestimate the client’s receive window. In our experiment, throughput fell from 1781.163 MB/s to 2.314 MB/s, a reduction of about 99.9%, while retransmissions remained at zero.

The inflate attack showed a different behaviour. Increasing the Window Scale did not immediately produce an effect because Linux automatically increased the client’s receive buffer. After the receive buffer was fixed at 65536 bytes, the expected retransmissions appeared.

The Slow Read attack was simpler and more predictable. With a server limited to 20 worker connections, 20 slow connections were enough to exhaust the available pool and make the health endpoint unavailable during the attack.

We also tested three defenses. Static ARP prevented the attacker from obtaining the MITM position. IPsec AH prevented the attacker’s packet modification pipeline from operating on the protected traffic. The live IDS detected the Window Scale mismatch.

An important part of this project was comparing expected results with actual results. The inflate experiment showed that an attack can be implemented correctly but still produce little visible damage when the environment has different TCP buffering and timing behaviour. This makes controlled testing and measurement important when studying TCP attacks.

13 References

- [1] RFC 7323, *TCP Extensions for High Performance*.

<https://www.rfc-editor.org/rfc/rfc7323>

Used for the TCP Window Scale option, its shift range, and the window-size calculation.

- [2] RFC 9293, *Transmission Control Protocol*.

<https://www.rfc-editor.org/rfc/rfc9293>

Used for TCP handshake, retransmission, window, and flow-control behaviour.

- [3] Fedora Magazine, *TCP window scaling, timestamps and SACK*.

<https://fedoramagazine.org/tcp-window-scaling-timestamps-and-sack/>

Used as a practical reference for Linux TCP Window Scale behaviour.

- [4] Qualys, *Are you ready for slow reading?*

<https://blog.qualys.com/vulnerabilities-threat-research/2012/01/05/slow-read>

Used as the design reference for the Slow Read attack.

- [5] `slowhttpptest` project.

<https://github.com/shekya/slowhttpptest>

Used as a reference for the Slow Read attack class. The actual experiment used a custom Python client rather than `slowhttpptest`.

- [6] CVE-2008-4609, Sockstress / TCP persist-timer DoS, National Vulnerability Database.

<https://nvd.nist.gov/vuln/detail/CVE-2008-4609>

Used as background information for TCP state-exhaustion attacks.